PUBLIC USAGE GUIDE

Native Runtime Package

Practical instructions for installation, Python integration, production startup, safe shutdown, updates, and first-line support.

What this package provides

The package supplies a compiled, GPU-accelerated detection runtime for use by a compatible host application. The public Python adapter selects the correct platform binary and provides the supported application interface.

Use the adapter	Import **movement_tracking.py**. Do not open the DLL or SO directly with ctypes or another language binding.

Quick start

1. Stop the host application before installing or updating runtime files.
2. Copy the complete supplied package and preserve every filename and subfolder.
3. Confirm that the production TensorRT engine exists as **models/model.engine**.
4. Start the application and wait until model loading reports that the runtime is ready.
5. Run one test image before enabling live-camera operation.

Document scope

This guide is intentionally focused on public deployment and supported use. It does not describe private implementation, build, or algorithm details.

System and package requirements

System	Architecture	Python	Runtime
Windows 10 or 11	x86-64	CPython 3.11, 64-bit	detection.dll
Linux	x86-64	CPython 3.10, 64-bit	detection.so

Hardware and software

- A compatible NVIDIA GPU and NVIDIA driver.
- The CUDA, TensorRT, and Python dependencies supplied for the host application.
- A compatible TensorRT detection engine named **model.engine**.
- Sufficient GPU memory and system RAM for the selected model and input resolution.

Primary package files

File	Purpose
movement_tracking.py	Supported Python adapter used by the host application.
native/windows-x86_64/detection.dll	Native runtime for supported Windows installations.
native/linux-x86_64/detection.so	Native runtime for supported Linux installations.
native/detection_settings.xml	Production detection configuration.
native/movement_tracking_manifest.json	Runtime release metadata.
models/model.engine	Fixed production model loaded during application startup.

Keep one release together Do not rename files, flatten folders, or combine files from different runtime releases. Leave any additional supplied release files unchanged.

Installation and startup

Install

1. Stop live detection and close the host application completely.
2. Copy the complete runtime package into the host application's root directory.
3. Preserve the supplied directory layout and filenames exactly.
4. Place the compatible TensorRT engine at **models/model.engine**.
5. Activate the supported Python environment for the operating system.
6. Start the application with its normal launcher or operating-system service.
7. Wait until startup reports that model loading and warm-up are complete.

Expected startup sequence

Stage	Expected result
Package selection	The adapter chooses the Windows or Linux binary automatically.
Model check	The fixed path models/model.engine is found and recognized as a compatible detection engine.
GPU initialization	The NVIDIA execution environment and engine context initialize successfully.
Warm-up	One or more initial passes prepare the model for normal inference.
Ready	The application continues startup and enables detection features.

First startup The first model load can take longer while GPU resources initialize. Do not start live operation until the ready state is reported.

Basic verification

- The process starts without a Python import or native loading exception.
- Model loading reaches the ready state.
- A known test image returns a valid result object.
- Live detection can start and stop normally.

Python integration

Import the public adapter

```
import movement_tracking
```

Load the fixed production model

```
import movement_tracking

def show_progress(percent, message):
print(f"{percent:3d}% | {message}")

movement_tracking.load_runtime_model(show_progress)
print(movement_tracking.status())
```

The host application normally loads the model once during startup. Repeated loading is unnecessary. Use **movement_tracking.status()** for diagnostics such as the loaded state, model path, selected device, and runtime availability.

Detect one image

```
import cv2
import movement_tracking

frame = cv2.imread("sample.png")
if frame is None:
raise RuntimeError("The sample image could not be read")

result = movement_tracking.detect(
frame, confidence=0.05, iou=0.45,
mode="balanced", encode_preview=True,
)

for item in result.get("detections", []):
print(item)

preview = movement_tracking.render_result_image()
```

Image format	Supply NumPy/OpenCV images in BGR order. Set encode_preview to false when the caller needs detection data only.

Operation and safe shutdown

Detection modes

Mode	Recommended use
fast	Lowest latency when throughput is the priority.
balanced	General production use and the recommended starting point.
accuracy	More detailed processing where additional latency is acceptable.

Use lower-case mode names. Performance-test the complete production workflow before changing from **balanced**. The host application may apply class-specific thresholds and movement rules in addition to the values passed in a basic detection call.

Safe shutdown order

Use the normal service stop command or press **Ctrl+C** once in the server console. The application should complete these stages before the process exits:

1. Stop live detection and reject new frames.
2. Allow active detection work to finish.
3. Unload the model and release GPU resources.
4. Release the camera.
5. Stop the application service.

Direct Python cleanup

```
try:
# Run the application workload.
pass
finally:
movement_tracking.unload_model()
```

Do not force termination	Wait for detection, GPU cleanup, and camera release to finish before closing the console, restarting, or powering off the computer.

Update procedure

1. Stop the application and back up the installed runtime package.
2. Replace the adapter, native folder, and related release files as one matching set.
3. Replace model.engine only when a compatible model is intentionally being deployed.
4. Restart, verify startup, test one image, and perform one controlled live-camera test.

Troubleshooting and support

Problem	Recommended action
Runtime file is missing	Restore the complete supplied package and preserve its directory structure.
Python import fails	Confirm the exact supported CPython version and 64-bit architecture.
Release validation fails	Reinstall one complete matching release. Do not combine old and new files.
model.engine is missing	Place the compatible production engine at the required fixed path.
Model is not a YOLO detection engine	Deploy the correct TensorRT detection engine for this runtime and GPU environment.
CUDA or TensorRT initialization fails	Check the NVIDIA driver, runtime dependencies, and GPU compatibility.
Detection returns no results	Verify the input image, model classes, confidence threshold, and detection settings.
Address or port is already in use	Stop the existing application instance before starting another one.
Shutdown appears slow	Wait for active detection work and GPU cleanup before forcing termination.

Information to provide for support

- Operating system and architecture.
- Python major/minor version and whether it is 64-bit.
- GPU model and installed NVIDIA driver version.
- Runtime release identifier and published file hashes.
- The exact startup error text and the time it occurred.

Protect operational data	Do not send credentials, private keys, production datasets, private source code, or unrelated system files.

Public distribution

Distribute the approved runtime, adapter, required configuration, public documentation, and published file hashes as one release. Do not rename or modify runtime files. Private source code, build tooling, debug symbols, generated build files, and internal engineering documents are not part of the public package.